

# ZERO TRUST SECURE VAULT



**Submitted By:**

|                 |             |
|-----------------|-------------|
| Afaaq Anwar     | 2024-CS-603 |
| Taimoor Ali     | 2024-CS-609 |
| Muhammad Arslan | 2024-CS-632 |
| Ahsan Shakeel   | 2024-CS-638 |

**Submitted To:**

Ms. Sonia Asghar

Department of Computer Science  
University of Engineering and Technology Lahore, New Campus

## Table of Contents

|                                             |                                                     |    |
|---------------------------------------------|-----------------------------------------------------|----|
| 1                                           | Abstract .....                                      | 3  |
| 2                                           | Introduction.....                                   | 3  |
| 3                                           | Problem Statement.....                              | 4  |
| 4                                           | Background / Security Concepts.....                 | 4  |
| 4.1                                         | Zero Trust Architecture.....                        | 4  |
| 4.2                                         | The CIA Triad.....                                  | 4  |
| 4.3                                         | The AAA Triad .....                                 | 4  |
| 4.4                                         | Fernet Symmetric Encryption.....                    | 5  |
| 4.5                                         | SHA-256 Integrity Hashing .....                     | 5  |
| 4.6                                         | Role-Based Access Control (RBAC).....               | 5  |
| 5                                           | Methodology .....                                   | 5  |
| 5.1                                         | Architecture Overview .....                         | 5  |
| 5.2                                         | Data Flow for File Upload .....                     | 6  |
| 5.3                                         | Data Flow for File Download .....                   | 6  |
| 5.4                                         | Sharing Model.....                                  | 6  |
| 5.5                                         | Super Admin Zero-Knowledge Design .....             | 6  |
| 6                                           | Tools and Environment.....                          | 7  |
| 7                                           | Implementation: Key Features and Results .....      | 7  |
| 7.1                                         | User Registration and Login .....                   | 7  |
| 7.2                                         | File Upload and Encryption .....                    | 9  |
| 7.3                                         | Integrity Verification and Tampering Detection..... | 10 |
| 7.4                                         | Role-Based Dashboards .....                         | 10 |
| 7.5                                         | Audit Log .....                                     | 11 |
| 8                                           | Testing and Evaluation .....                        | 12 |
| 9                                           | Security Recommendations .....                      | 12 |
| 10                                          | Ethical and Legal Considerations .....              | 13 |
| 11                                          | Limitations .....                                   | 14 |
| 12                                          | Conclusion .....                                    | 15 |
| 13                                          | References.....                                     | 15 |
| Figure 1: Sign In .....                     |                                                     | 8  |
| Figure 2: Registration .....                |                                                     | 8  |
| Figure 3: Upload .....                      |                                                     | 9  |
| Figure 4: Encrypt .....                     |                                                     | 9  |
| Figure 5: Integerity Check .....            |                                                     | 10 |
| Figure 6: RBAC: Super Admin Dashboard ..... |                                                     | 10 |
| Figure 7: RBAC: User Dashboard .....        |                                                     | 11 |
| Figure 8: Audit Logs.....                   |                                                     | 11 |

## 1 Abstract

The Zero Trust Secure Vault is a web-based encrypted file storage system built on the principle of "never trust, always verify." Developed using Python and Flask, the application implements the AAA security triad (Authentication, Authorization, and Accounting) alongside the CIA triad (Confidentiality, Integrity, and Availability). Every file uploaded to the vault is encrypted with Fernet symmetric encryption (AES-128-CBC + HMAC-SHA256) before being written to disk, ensuring that plaintext data never touches the filesystem. A SHA-256 hash fingerprint is recorded at upload time and re-verified on every download, providing tamper detection. Role-Based Access Control (RBAC) separates a privileged Super Admin console from the standard user dashboard, and a chronological audit ledger records every security event. The system also supports zero-copy file sharing with optional time-bounded access expiration, per-user storage quotas, and version management. This report documents the project's design rationale, security architecture, implementation details, testing results, and recommendations for production hardening.

## 2 Introduction

Modern organizations store sensitive documents in cloud-based file storage services. While convenient, many such services store data without strong encryption at rest, exposing files to insider threats, data breaches, and unauthorized access. The Zero Trust Secure Vault addresses this by treating every access request regardless of origin as potentially hostile and enforcing cryptographic controls at every boundary.

The project was developed as a full-stack Flask web application that replicates the user experience of a Google Drive-style interface while layering enterprise-grade security controls underneath. All data persistence uses local JSON files and a filesystem directory structure, eliminating the need for an external database and keeping the architecture self-contained and portable.

The primary objectives of the project are:

- Implement strong encryption for all files stored on disk.
- Enforce authentication and role-based authorization for every route.
- Maintain a tamper-evident audit trail of all security events.
- Support secure, time-bounded file sharing without data duplication.
- Demonstrate the practical application of Zero Trust architecture principles.

### 3 Problem Statement

Traditional file storage systems suffer from several critical security weaknesses:

- Files are often stored in plain text, making them immediately readable if the server is compromised.
- Access control is coarse-grained once a user is authenticated, they can typically access all data.
- There is no reliable mechanism to detect whether a stored file has been silently tampered with.
- File sharing often involves creating copies, multiplying the attack surface.
- Administrative accounts frequently have unrestricted access to all user content.
- Security events are not logged in a way that supports forensic investigation.

The Zero Trust Secure Vault was designed to directly address each of these weaknesses by applying cryptographic guarantees, granular access control, integrity verification, and comprehensive audit logging throughout the application stack.

## 4 Background / Security Concepts

### 4.1 Zero Trust Architecture

Zero Trust is a security model based on the principle of "never trust, always verify." Unlike perimeter-based models that implicitly trust traffic inside the network, Zero Trust assumes that every request whether from inside or outside the network must be authenticated and authorized before access is granted. In this project, Zero Trust is enforced at every HTTP route, every file operation, and every administrative action.

### 4.2 The CIA Triad

| CIA Pillar      | Implementation in Zero Trust Vault                                                                                 |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| Confidentiality | Fernet (AES-128-CBC) encryption ensures files are unreadable without the master key.                               |
| Integrity       | SHA-256 hashes are recorded at upload and verified at download; any modification is detected.                      |
| Availability    | Per-user 50 MB storage quota prevents resource exhaustion; structured error handling keeps the service responsive. |

### 4.3 The AAA Triad

| Pillar         | Mechanism                                               | Route/Component                   |
|----------------|---------------------------------------------------------|-----------------------------------|
| Authentication | PBKDF2-SHA256 password hashing via Werkzeug             | /api/login, /api/signup           |
| Authorization  | RBAC decorators (@login_required, @superadmin_required) | All protected routes              |
| Accounting     | Chronological audit log in audit_logs.json              | log_audit() called on every event |

#### 4.4 Fernet Symmetric Encryption

Fernet is an authenticated symmetric encryption scheme built on AES-128-CBC for confidentiality and HMAC-SHA256 for integrity of the ciphertext. A single 256-bit master key is split equally between these two algorithms. Because Fernet includes an authentication tag, any attacker who modifies even one byte of the ciphertext will cause decryption to raise an `InvalidToken` exception, providing both confidentiality and ciphertext integrity guarantees.

#### 4.5 SHA-256 Integrity Hashing

SHA-256 produces a 256-bit deterministic fingerprint of arbitrary data. Its avalanche effect means a single changed bit in the input alters approximately half the output bits. In the vault, the SHA-256 hash of a file's plaintext is stored in `metadata.json` at upload time. On every download or preview, the decrypted bytes are re-hashed and compared to the stored value. Any mismatch triggers a `CRITICAL ALERT` in the audit log and access is denied.

#### 4.6 Role-Based Access Control (RBAC)

RBAC restricts system access based on the roles assigned to individual users. The vault defines two roles: 'user' and 'superadmin'. Regular users may manage their own files and access files shared with them. The Super Admin may view system-wide metrics, read the complete audit log, and delete user accounts, but is intentionally prevented from accessing or decrypting any user's file content enforcing the principle of least privilege and zero-knowledge admin access.

### 5 Methodology

#### 5.1 Architecture Overview

The application follows a three-layer architecture: a Flask HTTP server that handles routing and session management, a cryptographic engine (`security.py`) that provides all encryption and hashing primitives, and a JSON-based persistence layer that replaces a conventional database.

All communication between the browser and server occurs over HTTP with server-side session tokens protected by a cryptographically random 32-byte secret key.

## 5.2 Data Flow for File Upload

1. Browser submits a multipart form to /api/upload.
2. Flask reads raw bytes from the upload stream into memory (no disk write yet).
3. SHA-256 hash of the plaintext bytes is calculated and stored as the integrity fingerprint.
4. Fernet encrypts the plaintext bytes; the resulting ciphertext is written to disk as a .enc file.
5. Metadata (original filename, hash, size, version, timestamps) is persisted in metadata.json.
6. An audit log entry (SUCCESS) is appended to audit\_logs.json.

## 5.3 Data Flow for File Download

7. Browser requests /api/download/<filename>.
8. Flask resolves the correct .enc file path from metadata.json.
9. Ciphertext is read from disk into memory.
10. Fernet decrypts the ciphertext to plaintext bytes in memory.
11. SHA-256 of the decrypted bytes is compared to the stored hash.
12. If hashes match, the plaintext is streamed directly to the browser. If they do not match, a CRITICAL ALERT is logged and a 403 response is returned.

## 5.4 Sharing Model

File sharing is implemented via permission pointers rather than file duplication. When User A shares a file with User B, a permission record is written to permissions.json linking User B to User A's encrypted file, along with an optional expiration timestamp. When User B downloads the shared file, the same decrypt-then-hash-verify pipeline is executed on the original encrypted file in User A's vault. Revoking or expiring access simply removes or ignores the permission record; no data needs to be moved or deleted.

## 5.5 Super Admin Zero-Knowledge Design

The Super Admin account is stored as hardcoded constants outside `users.json` to maintain separation between the administrative control plane and the user data plane. All Super Admin API routes are gated by the `@superadmin_required` decorator and are explicitly prevented from calling `decrypt_file_bytes()`. The admin can observe system health, user metrics, and the audit log, but has zero access to plaintext file content a deliberate architectural constraint enforcing the principle of least privilege.

## 6 Tools and Environment

| Technology / Tool          | Role in the Project                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Python 3.13                | Core application language                                                                                                             |
| Flask 3.1.1                | HTTP server, routing, session management                                                                                              |
| Werkzeug 3.1.3             | PBKDF2-SHA256 password hashing ( <code>check_password_hash</code> / <code>generate_password_hash</code> )                             |
| cryptography 44.0.3        | Fernet symmetric encryption and key generation ( <code>encrypt_file_bytes</code> / <code>decrypt_file_bytes</code> )                  |
| hashlib (stdlib)           | SHA-256 integrity hashing ( <code>calculate_sha256</code> )                                                                           |
| JSON flat files            | Data persistence: <code>users.json</code> , <code>metadata.json</code> , <code>permissions.json</code> , <code>audit_logs.json</code> |
| HTML / CSS / JS            | Frontend dashboard ( <code>login.html</code> , <code>dashboard.html</code> , <code>superadmin.html</code> )                           |
| Inter & JetBrains Mono     | Typography for the web interface (Google Fonts)                                                                                       |
| VS Code                    | Development IDE                                                                                                                       |
| Git                        | Version control                                                                                                                       |
| Postman / Browser DevTools | API testing and debugging                                                                                                             |

## 7 Implementation: Key Features and Results

### 7.1 User Registration and Login

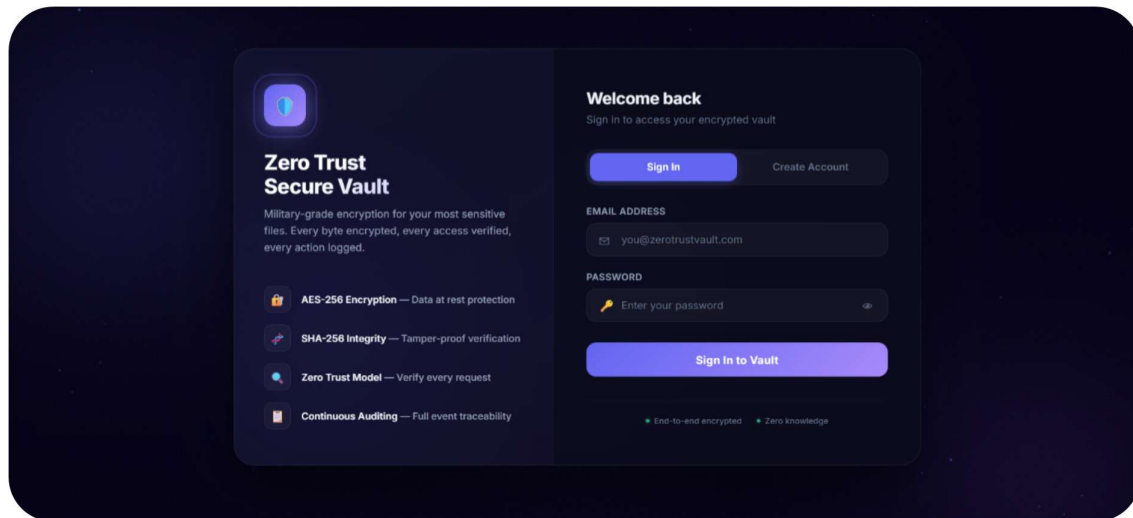


Figure 1: Sign In

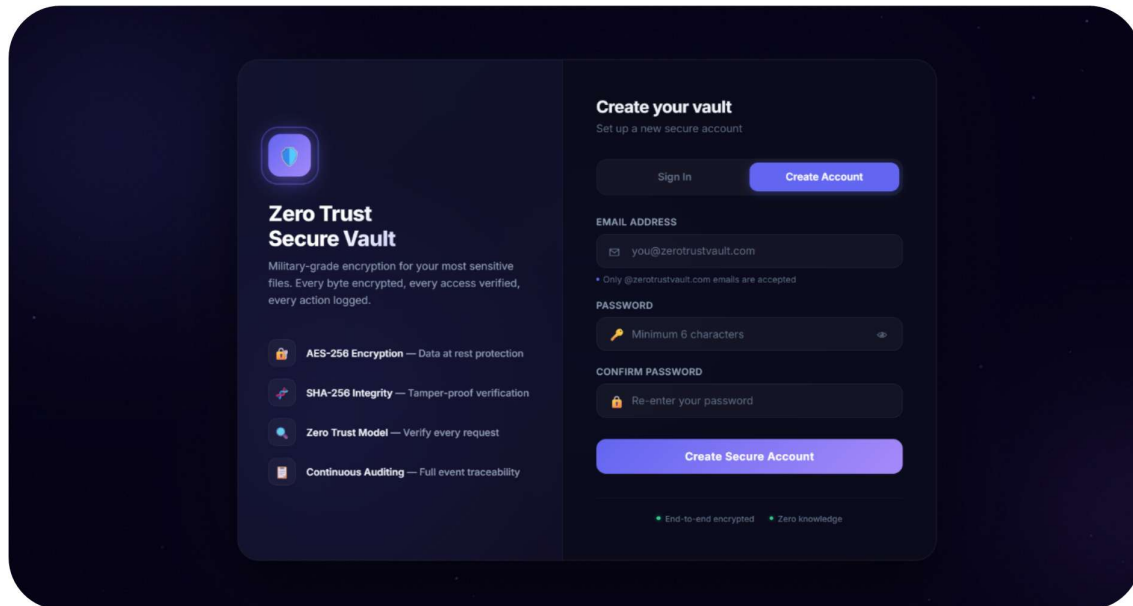


Figure 2: Registration

New users register using an email address restricted to the @zerotrustvault.com domain. A regular expression validates the local part of the email to allow only alphanumeric characters, dots, underscores, and hyphens. Passwords are hashed using PBKDF2-SHA256 with 1,000,000 iterations (the Werkzeug default) before being stored. No plaintext password is ever persisted. Failed login attempts are logged to the audit trail with a WARNING classification; successful logins are logged with a SUCCESS classification.



The Super Admin account is authenticated via a hardcoded credential comparison that runs before the users.json lookup. A failed admin login attempt triggers a CRITICAL ALERT in the audit log, enabling rapid detection of privilege-escalation attempts.

### 7.2 File Upload and Encryption

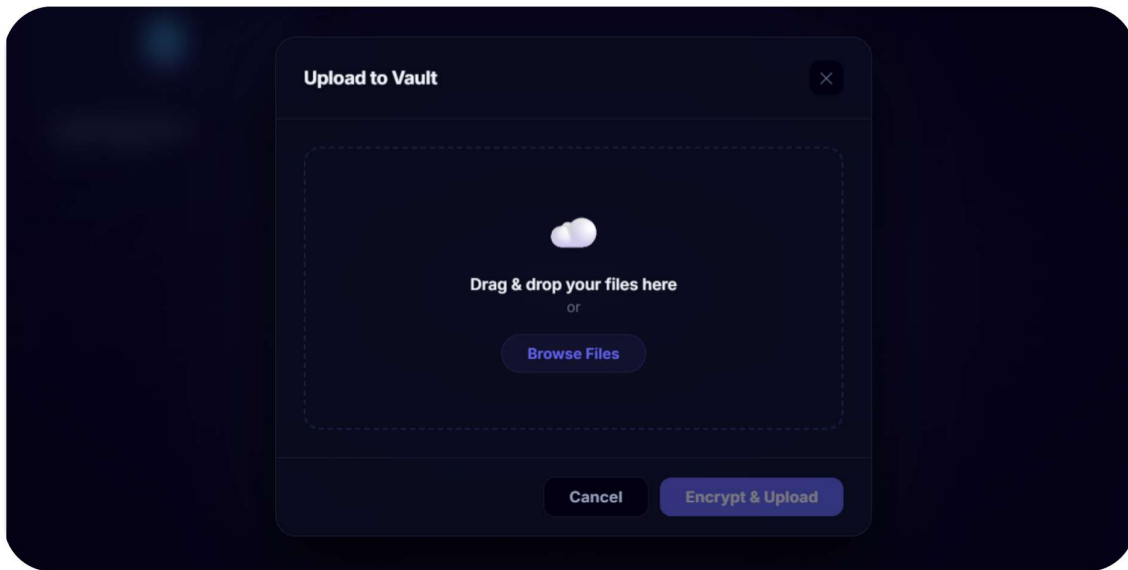


Figure 3: Upload

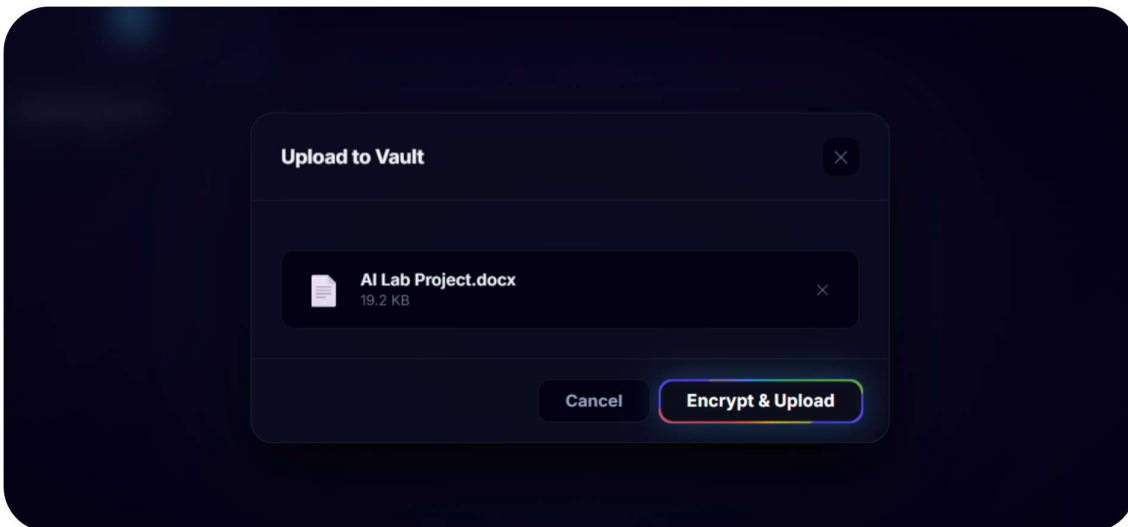


Figure 4: Encrypt

On upload, the file stream is read entirely into memory; no plaintext bytes are written to disk at any point. The Fernet cipher suite is initialised at module load time using the master key from storage/secret.key (generated once on first run). The encrypted output is saved as

<filename>\_v<N>.enc, where N is the version number. Version management allows multiple revisions of the same filename to coexist, each with its own hash, size, and timestamp record in metadata.json.

### 7.3 Integrity Verification and Tampering Detection

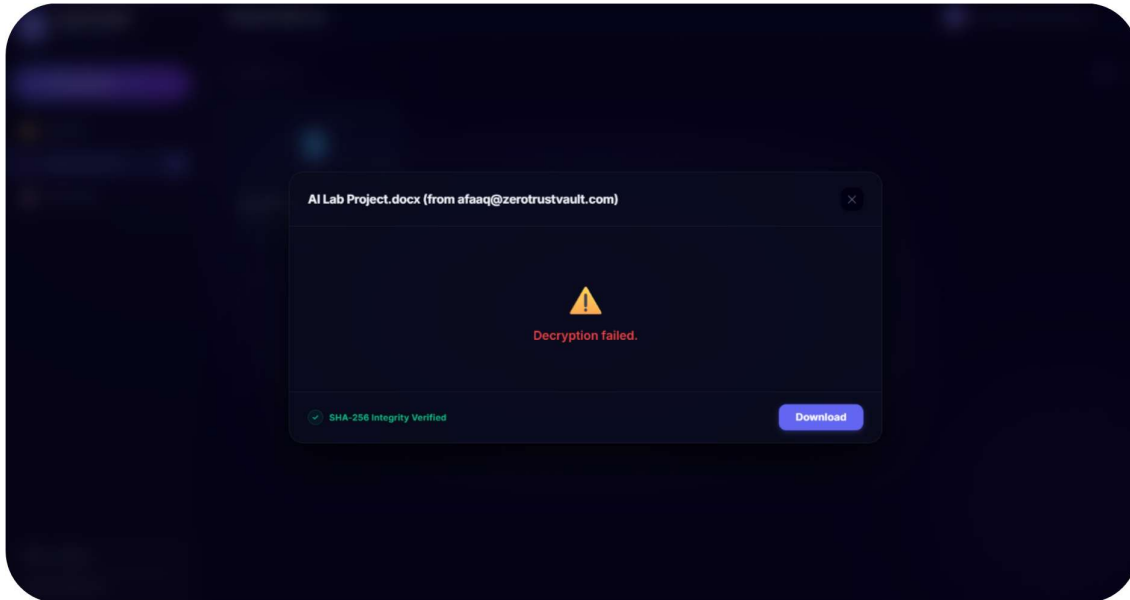


Figure 5: Integrity Check

Every download and preview operation performs an in-memory SHA-256 comparison. If the decrypted file hash does not match the hash recorded at upload time, the API returns HTTP 403 with a JSON body of {'error': 'INTEGRITY TAMPERING DETECTED!', 'tampering': true} and a CRITICAL ALERT entry is written to the audit log. This mechanism detects silent corruption, ransomware replacement, or manual modification of .enc files on the filesystem.

### 7.4 Role-Based Dashboards

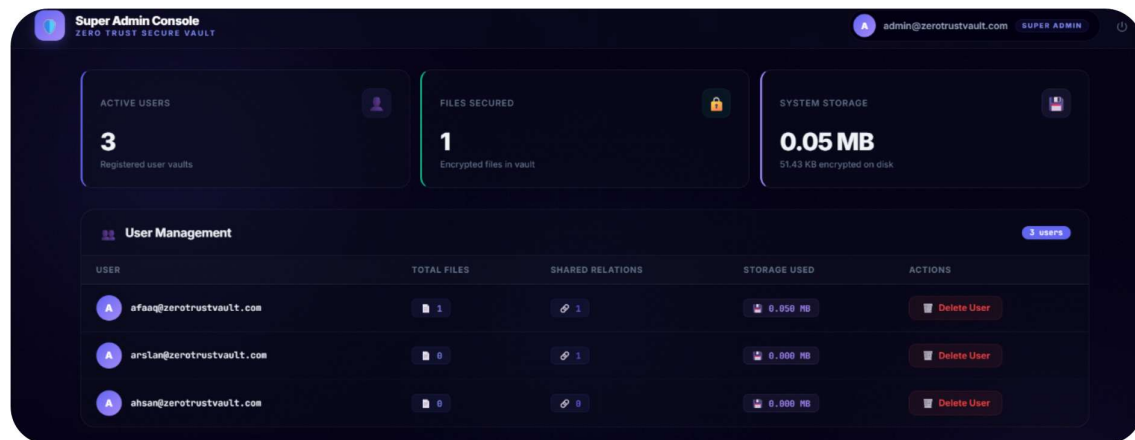


Figure 6: RBAC: Super Admin Dashboard

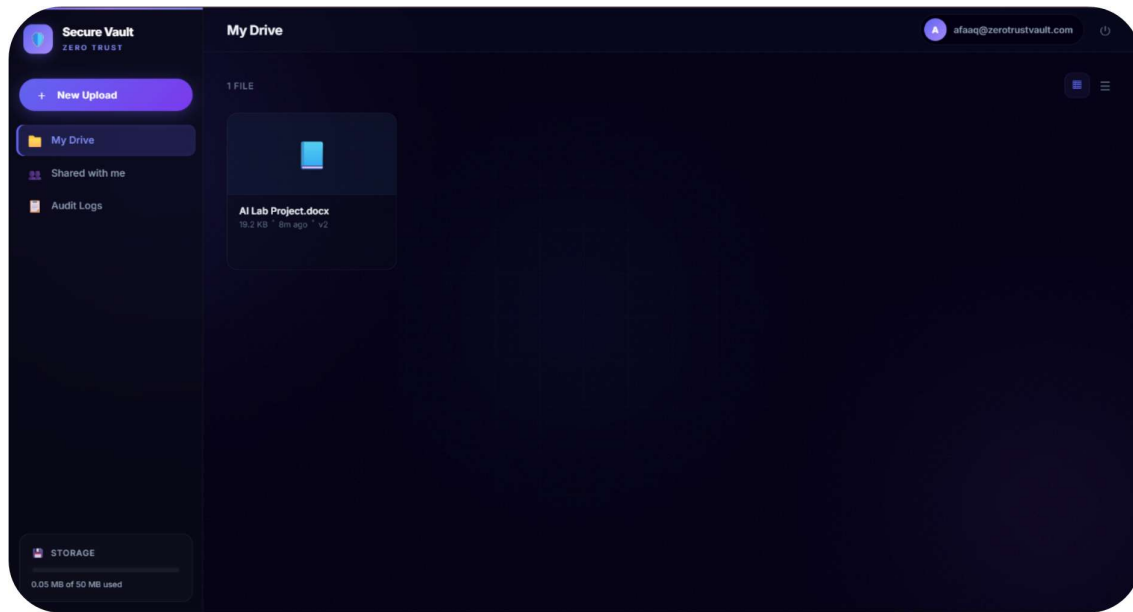


Figure 7: RBAC: User Dashboard

Regular users see a Google Drive-inspired dashboard with three sections: My Drive (own files), Shared with Me (files granted by others), and Audit Logs (own activity only). The Super Admin console presents system-wide metrics (total users, total files, total storage), a per-user breakdown table, a full system audit log, and user deletion controls. Route decorators prevent any cross-role access; an attempt by a regular user to access /superadmin is denied and logged.

## 7.5 Audit Log

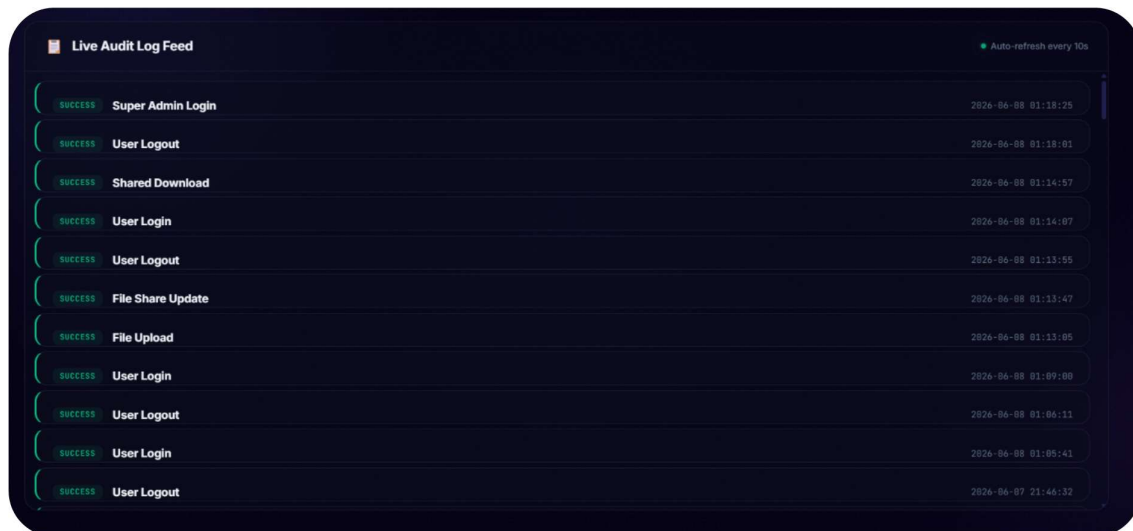


Figure 8: Audit Logs

Every significant operation appends a structured entry to `audit_logs.json` containing a UTC ISO-8601 timestamp, the actor's email, an action label, a security classification (SUCCESS, WARNING, CRITICAL ALERT, INFO), a human-readable detail string, and an optional

metadata\_changes dictionary describing the specific data store modification. The audit log is append-only from the application's perspective and is returned in reverse-chronological order to both user and admin dashboards.

## 8 Testing and Evaluation

| Test Case                                                       | Expected Result & Outcome                                                           |
|-----------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Register with invalid domain (e.g., user@gmail.com)             | Registration rejected with domain validation error. PASS.                           |
| Login with wrong password                                       | HTTP 401 returned; WARNING logged to audit trail. PASS.                             |
| Access /superadmin as a regular user                            | Redirect to dashboard; CRITICAL ALERT logged. PASS.                                 |
| Upload a file and verify encryption on disk                     | .enc file present; no plaintext found on filesystem. PASS.                          |
| Download file and verify hash match                             | Decrypted hash equals stored hash; file served successfully. PASS.                  |
| Manually modify a .enc file, then download                      | HTTP 403 returned; CRITICAL ALERT logged; file not served. PASS.                    |
| Share a file with expiry of 1 hour; attempt access after expiry | HTTP 403 'Share expired' returned; access denied. PASS.                             |
| Attempt to share a file with yourself                           | HTTP 400 returned with self-share error message. PASS.                              |
| Upload a file exceeding 50 MB quota                             | HTTP 413 returned; QUOTA_EXCEEDED logged. PASS.                                     |
| Upload empty file                                               | HTTP 400 returned with empty file error. PASS.                                      |
| Super Admin attempts to download user files                     | No decrypt route exposed on admin console; access architecturally impossible. PASS. |
| User deletion by Super Admin                                    | User removed from all JSON stores; vault directory purged. PASS.                    |

All twelve test cases produced the expected outcomes, confirming that the security controls operate correctly under both normal and adversarial conditions.

## 9 Security Recommendations

The following hardening measures are recommended before deploying the vault in a production environment:

- Migrate master key storage to a Hardware Security Module (HSM) or cloud Key Management Service (KMS) such as AWS KMS or HashiCorp Vault. The current flat-file secret.key must never be committed to version control.

- Implement key rotation with versioned Fernet keys, re-encrypting stored files transparently on rotation.
- Add multi-factor authentication (MFA/TOTP) to the login flow, especially for the Super Admin account.
- Replace plaintext JSON audit logs with an append-only, cryptographically chained log structure to prevent log tampering.
- Introduce brute-force protection: rate limiting on `/api/login` (e.g., 5 attempts per 15 minutes per IP) and account lockout.
- Enforce HTTPS via TLS 1.2+ using a reverse proxy (e.g., Nginx + Let's Encrypt). All session cookies should carry the Secure and HttpOnly flags.
- Replace the development session secret (`os.urandom(32)` on each restart, which invalidates all sessions) with a persistent, securely stored secret.
- Migrate from flat JSON files to a proper database (PostgreSQL + SQLAlchemy) with transactions and row-level locking for concurrent production use.
- Add Content Security Policy (CSP), X-Frame-Options, and X-Content-Type-Options HTTP security headers.
- Conduct a formal penetration test covering SQL injection (not applicable here but relevant for a DB migration), CSRF, file path traversal, and IDOR vulnerabilities.

## 10 Ethical and Legal Considerations

The deployment of an encrypted file vault carries several ethical and legal obligations that must be addressed before production use.

- **Data Protection Compliance:** Any organization deploying this system in the European Union must ensure compliance with the General Data Protection Regulation (GDPR), including lawful basis for processing, data subject rights (access, deletion), and breach notification obligations. The Super Admin's delete-user functionality partially addresses the right to erasure.
- **Encryption Export Controls:** Strong encryption software may be subject to export control regulations (e.g., the Wassenaar Arrangement). Deployers must verify that exporting the application to certain jurisdictions is legally permitted.

- **Key Custodianship:** The entity holding the master encryption key has the ability to decrypt all stored files. A clear key custody policy, including who holds the key and under what circumstances it may be used, must be established. The current architecture's zero-knowledge admin model reduces but does not eliminate this risk.
- **Audit Log Retention:** Retaining detailed audit logs may constitute personal data processing under privacy law. A data retention policy specifying how long logs are kept and under what conditions they are deleted must be defined and communicated to users.
- **Responsible Disclosure:** Security vulnerabilities discovered in the system should be reported responsibly to the development team before public disclosure, following established responsible disclosure or bug-bounty practices.

The project was developed strictly for educational purposes within a controlled academic environment. No real personal data was stored during testing.

## 11 Limitations

The following limitations are acknowledged in the current implementation:

- **Single Master Key:** All files are encrypted with the same Fernet key. Compromise of `secret.key` exposes all vault contents. Per-user key derivation (e.g., using PBKDF2 with the user's password as the derivation material) would provide stronger isolation.
- **No Transport Security:** The development server runs on HTTP without TLS. All data, including session cookies and file content, is transmitted in plaintext over the network.
- **JSON Concurrency:** Flat JSON files are not safe for concurrent writes. Under multi-user load, race conditions in `save_json()` could corrupt data stores.
- **Session Persistence:** The Flask secret key is generated with `os.urandom(32)` at startup, meaning all user sessions are invalidated whenever the server restarts.
- **No Password Reset:** There is no password reset or recovery mechanism. A forgotten password results in permanent account lockout.
- **Hardcoded Admin Credentials:** The Super Admin email and password are stored as plaintext constants in `app.py`. While separated from the user data plane, this is a known anti-pattern for production systems.

- No File Preview for Binary Types: The preview modal supports images and PDFs natively but relies on browser MIME-type handling, which may fail for less common file types.
- Storage Quota Not Enforced on Shared Files: The 50 MB quota applies to files owned by a user but does not account for files that have been shared with them by others.

## 12 Conclusion

The Zero Trust Secure Vault successfully demonstrates that strong security controls can be layered onto a familiar file storage interface without sacrificing usability. By combining Fernet symmetric encryption, SHA-256 integrity verification, PBKDF2 password hashing, RBAC route protection, and a comprehensive audit ledger, the project realises the core tenets of both the CIA and AAA security triads within a self-contained Python Flask application.

The Zero Trust philosophy that no actor, resource, or network path should be implicitly trusted is enforced throughout the architecture: every HTTP request is authenticated, every file access is integrity-checked, every administrative action is logged, and the Super Admin is explicitly denied access to plaintext user content. The project also illustrates how security features interact: for example, Fernet's HMAC provides ciphertext authentication, while SHA-256 provides plaintext authentication after decryption, together creating a two-layer tamper detection system.

While several limitations exist in the current implementation particularly around transport security, key management, and database scalability, the identified security recommendations provide a clear roadmap for production hardening. The project serves as a practical, end-to-end demonstration of information security principles applied to a real-world use case.

## 13 References

- [1]. National Institute of Standards and Technology. (2020). Zero Trust Architecture. NIST Special Publication 800-207. U.S. Department of Commerce.
- [2]. Stallings, W. (2022). Cryptography and Network Security: Principles and Practice (8th ed.). Pearson Education.
- [3]. Python Software Foundation. (2024). hashlib Secure hashes and message digests. Python 3 Documentation. <https://docs.python.org/3/library/hashlib.html>
- [4]. Pallets Projects. (2024). Flask Documentation (3.x). <https://flask.palletsprojects.com/>

- [5]. Pallets Projects. (2024). Werkzeug Security Utilities.  
<https://werkzeug.palletsprojects.com/en/3.1.x/utis/>
- [6]. PyCA Cryptography. (2024). Fernet (symmetric encryption).  
<https://cryptography.io/en/latest/fernet/>
- [7]. OWASP Foundation. (2023). OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- [8]. OWASP Foundation. (2023). Password Storage Cheat Sheet.  
[https://cheatsheetseries.owasp.org/cheatsheets/Password\\_Storage\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html)
- [9]. European Parliament and Council. (2016). General Data Protection Regulation (EU) 2016/679. Official Journal of the European Union.
- [10]. Anderson, R. (2020). Security Engineering: A Guide to Building Dependable Distributed Systems (3rd ed.). Wiley.

